

Bike Sharing Locale – DDD e Disegno Architettuale

Obiettivo: modello semplice ma logicamente robusto per assegnare biciclette a utenti registrati, senza pagamenti né prenotazioni.

Cliente	Comune (servizio bike sharing non commerciale)
Scope	Flotta biciclette + assegnazioni temporanee + query operative
Fuori scope	Pagamenti, stazioni, prenotazioni, manutenzione avanzata, GPS
Qualità richiesta	Coerenza, assenza conflitti, regole di dominio chiare e testabili
Data / Versione	28/01/2026 – v1.1

1. Problem Space – Business Capabilities

Il Comune vuole evitare incoerenze: una bici non può essere assegnata a più persone e un utente non può avere più di una bici attiva. Queste regole sono la sostanza del servizio.

Capability	Ufficio logico	Responsabilità	Confine (cosa NON fa)
Gestione biciclette	Fleet Office	Registra elenco biciclette (codice univoco, attiva/disattiva)	Non assegna bici
Gestione utenti	Identity Office	Registra utenti (id univoco, dati minimi)	Non decide assegnazioni
Assegnazione temporanea	Assignment Office (CORE)	Crea/chiude assegnazioni applicando invarianti	Non gestisce anagrafiche
Consultazione operativa	Read/Query Office	Viste: bici in uso, bici per utente	Non modifica stato

Criteri architetturali:

- Alta coesione: ogni ufficio ha un solo motivo di cambiamento.
- Basso accoppiamento: il CORE usa solo identificativi (UserId, BicycleId).

Le business capabilities sono separate perché rispondono a **motivi di cambiamento diversi**: la gestione delle biciclette riguarda le **risorse**, la gestione degli utenti riguarda le **identità**, mentre l'assegnazione è l'unico punto in cui si applicano le **regole di esclusività**. Questa separazione garantisce **alta coesione** e **basso accoppiamento**, evitando che regole di dominio si disperdano nelle anagrafiche.

2. Sottodomini e priorità

Sottodominio	Tipo	Perché	Esternalizzabile?
Assegnazioni temporanee	Core Domain	Regole specifiche: esclusività e coerenza	No
Registro biciclette	Supporting	Supporta il core, regole semplici	Sì
Registro utenti	Supporting	Supporta il core, regole semplici	Sì
AuthN/AuthZ, logging, monitoring	Generic	Infrastruttura standard	Sì

Il valore principale del sistema nasce nel sottodominio di **Assegnazione temporanea**, perché è lì che vengono applicate le regole che evitano conflitti e incoerenze. I sottodomini di gestione utenti e biciclette sono di supporto e **potenzialmente esternalizzabili**, poiché non contengono logica distintiva del servizio comunale.

3. Solution Space – Bounded Context e Linguaggio

Bounded Context per separare significati e regole. Termini principali per contesto:

Bounded Context	Ubiquitous Language	Oggetti	Significato
Fleet	Bicicletta, CodiceBici, Flotta	Bicycle, BicycleId	Bici = risorsa inventariata
Identity	Utente, IdUtente	User, UserId	Utente = identità registrata
Assignment (CORE)	Assegnazione, Attiva, Rilascio	Assignment, Period	Relazione temporanea con invarianti
Query (Read)	Bici in uso, Bici per utente	Read models	Viste derivate dal CORE

Esempio di cambio significato:

- “Bicicletta” in Fleet = entità dell’inventario.
- “Bicicletta” in Assignment = BicycleId dentro un’assegnazione (conta per i vincoli).
Il bounded context serve a evitare ambiguità semantiche (il significato condiviso dei dati): lo stesso termine può avere significati diversi a seconda del contesto.
Ad esempio, *Bicicletta* nel contesto Fleet rappresenta una risorsa inventariata, mentre nel contesto Assignment è solo un identificativo utilizzato per applicare vincoli di dominio.

4. Invarianti di dominio:

Le invarianti di dominio rappresentano regole **non negoziabili** il cui scopo è prevenire stati incoerenti del sistema. La violazione di una singola invariante comporterebbe situazioni ambigue, come doppie assegnazioni o utenti con più biciclette attive.

Eccoli:

1. Una bicicletta può avere al massimo una assegnazione attiva alla volta.
2. Un utente può avere al massimo una assegnazione attiva alla volta.
3. Non si crea un’assegnazione se l’utente non esiste.
4. Non si crea un’assegnazione se la bicicletta non esiste.
5. Il rilascio chiude una assegnazione attiva.
6. Un’assegnazione attiva ha startAt valorizzato ed endAt assente; endAt solo in chiusura.

5. Modello di dominio – Essenziale

Elemento	Tipo	Ruolo
Bicycle	Entity	Risorsa del Comune
User	Entity	Utente registrato
Assignment	Aggregate Root	User↔Bicycle nel tempo
BicycleId, UserId, AssignmentId	Value Object	Identità immutabili

Period (startAt, endAt?)	Value Object	Intervallo assegnazione
Status (ACTIVE/CLOSED)	Enum/VO	Stato esplicito

Scelta: lo stato “in uso” della bici è derivato dalle assegnazioni attive (single source of truth).

Lo stato “in uso” non è modellato come attributo della bicicletta, ma come proprietà della **relazione di assegnazione**. In questo modo si evita la duplicazione dell’informazione e si mantiene una **single source of truth (Golden record)** nel Core Domain.

6. Azioni (Use case)

Use case	Scopo	Regole
AssignBicycle	Assegnare una bici a un utente	No conflitti + esistenza
ReleaseBicycle	Rilasciare la bici	Solo assegnazione attiva
ListBikesInUse	Elenco bici in uso	Derivato da assegnazioni attive
GetUserBike	Bici assegnata a un utente	Derivato da assegnazioni attive

7. Struttura implementativa (no DB)

4 livelli: Domain (regole), Application (use case), Infrastructure (adattatori), API (HTTP). Il core non dipende dall'API: cambia l'interfaccia senza riscrivere le regole.

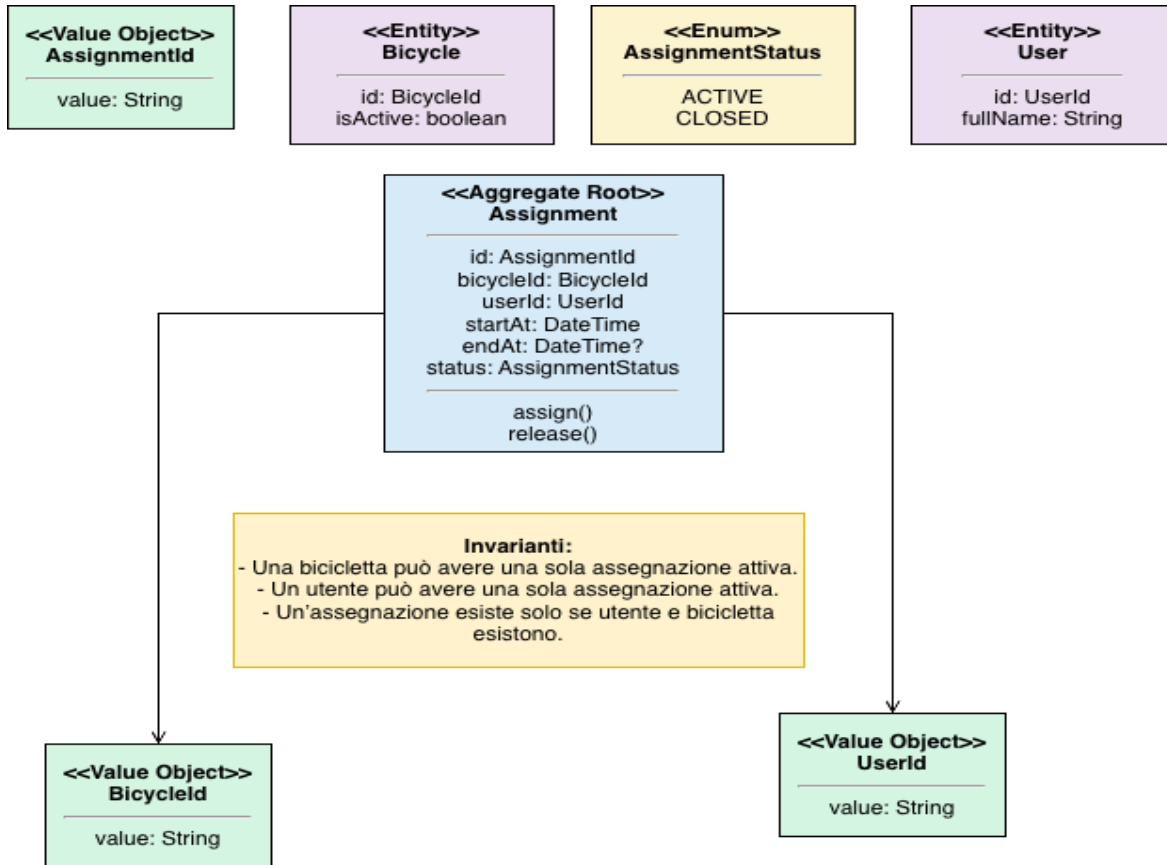
```
src/
  bike_sharing/
    domain/
      # Modello + invarianti (zero framework)
      models.py          # Bicycle, User, Assignment, VO
      policies.py        # Invarianti e regole di validazione
      errors.py          # Errori di dominio (Conflict, NotFound, ...)
      events.py          # (opz.) eventi di dominio
    application/
      # Orchestrazione dei casi d'uso
      ports.py           # Interfacce repository + clock + publisher
      use_cases.py       # Assign/Release/ListInUse/GetUserBike
    infrastructure/
      # Implementazioni concrete (oggi in-memory)
      in_memory_repos.py
      in_memory_publisher.py
    api/
      # Adattatore FastAPI (solo I/O)
      main.py            # wiring + routing
      schemas.py         # DTO request/response
    tests/
      test_domain.py
      test_use_cases.py
```

Motivazione:

- Il dominio contiene il valore (invarianti) ed è indipendente.
- L’application layer implementa azioni esplicite e testabili.
- Infrastructure può cambiare (DB, message bus) senza impattare il dominio.
- API è un adattatore: traduce HTTP↔use case, niente business logic.

8. Roadmap microservizi + Docker e ER:

- Parti con monolite modulare. Quando serve, estrai Assignment come primo microservizio (è il CORE).
- Fleet e Identity diventano servizi di supporto o integrazioni.
- Docker: un container per servizio; compose per sviluppo locale; healthcheck e log su stdout.



Il modello realizzato concentra le regole nel punto dove nasce il valore, mantenendo le anagrafiche semplice (come richiesto) e riducendo il rischio di incoerenze.